Chapter 7

Synthesis of Multilevel Circuits

Multilevel realizations are the preferred means of implementing combinational logic in *very large scale integrated* (VLSI) systems today. Because of the increased potential for reusing subcircuits, there are more degrees of freedom in implementing a Boolean function than in the two-level *programmable logic array* (PLA) case. However, this increased freedom also makes the task for automatically synthesizing the logic at a level competitive with manual design difficult.

The area of multilevel logic synthesis has blossomed since the mid-1980s. Many of the methods and algorithms developed thus far have been successfully used in commercially available computer-aided design packages. Two basic approaches have been adopted in multilevel logic synthesis. In the late 1970s, the LSS system was developed at IBM [21] using rule-based local transformations. The local transformation and rule-based methods use a set of rules which are fired when certain patterns are found in the network of logic gates. A rule transforms a pattern for a local set of gates and interconnections into another equivalent one. The transformations have limited optimization capability since they are local in nature and do not have a global perspective of the design. The LSS system has continued to evolve and now contains methods for global optimization (e.g., [9, 40]).

Beginning about 1981, in parallel with activity in two-level logic synthesis and influenced by it, an approach evolved that was based on algorithmic transformations. The algorithmic point of view uses two phases: a technology-independent step based on algorithms

for manipulating general Boolean functions [15] and a technology mapping step where the design described in terms of generic Boolean functions is mapped into a set of gates that can be implemented in the design method of choice (gate arrays, standard cells, or macrocells). Both rule-based methods (e.g., [8, 20]) and algorithmic methods (e.g., [11, 13]) have been successful.

Most algorithmic logic synthesis systems use algebraic [15, 16] and Boolean [11, 22] operations in the technology-independent phase of the design, and use graph covering methods pioneered in [28] for the technology mapping phase. We will focus on the algorithmic methods for logic synthesis in this chapter, since the logic transformations used in these approaches have been the central focus of synthesis for testability strategies.

We will describe the various logic transformations used in algorithmic logic synthesis systems in Section 7.1. Implementation details of the algorithms presented here can be found in [13, 14]. We will describe the notion of division and common divisors in Section 7.2 and describe algebraic and Boolean division algorithms in Sections 7.3 and 7.5, respectively. Internal "don't-care" conditions in multilevel logic circuits are described in Section 7.6. Finally, in Sections 7.7 and 7.8 we describe algorithmic approaches to technology mapping targeting an area cost function.

In this chapter, we will focus primarily on describing the available suite of logic transformation and restructuring methods. These methods can typically be used directly for area minimization. In Chapter 8, we will focus on delay minimization.

7.1 Logic Transformations

The goal of multilevel logic optimization is to obtain a representation of the Boolean function that is optimal with respect to area, speed, testability, or power dissipation. In order to restructure a logic function the operations described in the sequel are used.

7.1.1 Decomposition

Decomposition of a Boolean function is the process of reexpressing a single function as a collection of new functions. For example, the process of translating the expression

$$F = a \cdot b \cdot c + a \cdot b \cdot d + \overline{a} \cdot \overline{c} \cdot \overline{d} + \overline{b} \cdot \overline{c} \cdot \overline{d}$$

to the set of expressions

$$\begin{aligned} F &= X \cdot Y + \overline{X} \cdot \overline{Y} \\ X &= a \cdot b \\ Y &= c + d \end{aligned}$$

is decomposition.

7.1.2 Extraction

Extraction, related to decomposition, is applied to many functions. It is the process of identifying and creating some intermediate functions and variables, and reexpressing the original functions in terms of the original as well as the intermediate variables. Extraction creates nodes with multiple fan-outs. For example, extraction applied to the following three functions

$$\begin{aligned} F &= (a+b) \cdot c \cdot d + e \\ G &= (a+b) \cdot \overline{e} \\ H &= c \cdot d \cdot e \end{aligned}$$

yields

$$\begin{aligned} F &= X \cdot Y + e \\ G &= X \cdot \overline{e} \\ H &= Y \cdot e \\ X &= a+b \\ Y &= c \cdot d \end{aligned}$$

The operation thus identifies common subexpressions among different logic functions forming a network. New nodes corresponding to the common subfunctions are created and each of the logic functions in the original network is simplified as a result of the introduction of these new nodes. The optimization problem associated with the extraction operation is to find a set of intermediate functions such that the resulting network has minimum area, delay, power dissipation, or maximum testability.

7.1.3 Factoring

Like the sum-of-products, factored forms are a way of representing Boolean functions and are perhaps a more natural way for multilevel circuits than the sum-of-products representation. A factored form is a parenthesized representation of a tree network where each internal

node is an AND or an OR gate and each leaf is a literal. Factoring is the process of deriving a factored form from a sum-of-products representation of a function. For example,

$$F = a \cdot c + a \cdot d + b \cdot c + b \cdot d + e$$

can be factored to

$$F = (a+b) \cdot (c+d) + e$$

The optimization problem associated with factoring is to find a factored form with the minimum number of literals.

7.1.4 Substitution

Substitution, also called resubstitution, of a function G into F is the process of reexpressing F as a function of its original inputs and G. For example, substituting

$$G = a + b$$

into

$$F = a + b \cdot c$$

produces

$$F = G \cdot (a + c)$$

This operation creates an arc in the Boolean network connecting the node of the substituting function, namely G, to the node of the function being substituted into, namely F.

7.1.5 Elimination

Elimination, collapsing, or flattening is the the inverse operation of substitution. If G is a fan-in of F, collapsing G into F reexpresses F without G. It undoes the operation of substituting G into F. For example, if

$$\begin{aligned} F &= G \cdot a + \overline{G} \cdot b \\ G &= c + d \end{aligned}$$

then, collapsing G into F results in

$$\begin{aligned} F &= a \cdot c + a \cdot d + b \cdot \overline{c} \cdot \overline{d} \\ G &= c + d \end{aligned}$$

If the node G is not a primary output and does not fan-out to other nodes, then it may be removed from the Boolean network, resulting in a network with one less node.

Flattening a logic function could result in an exponentially larger representation. For example, consider the flattening of the nodes g_1 through g_k into F below.

$$\begin{aligned} F &= g_1 \cdot g_2 \cdots \cdot g_k \\ g_1 &= a_1 + b_1 \\ g_2 &= a_2 + b_2 \\ &\vdots \\ g_k &= a_k + b_k \end{aligned}$$

This will result in a representation for F with 2^k product terms. Functions that have manageable[1] sum-of-products representations are termed *flattenable* functions. Functions like F with large k above that do not have manageable sum-of-products representations are called *nonflattenable.*

7.2 Division and Common Divisors

In optimizing logic functions it is important to define operations which, when given functions f and p, find functions q and r such that $f = p \cdot q + r$, if such q and r exist. This operation is called the division of f by p generating quotient q and remainder r.

The function p is called a divisor of f if r is not null and a factor if r is null. For a given division operation the resulting q and r may be dependent upon the particular representation of f and p.

Some properties of divisors and factors are presented using the following propositions.

Proposition 7.2.1 *A logic function g is a Boolean factor of a logic function f if and only if $f \cdot \overline{g} = \phi$ (in other words, the ON-set of f is contained in the ON-set of g).*

Proposition 7.2.2 *If $f \cdot \overline{g} \neq \phi$, then g is a Boolean divisor of f.*

[1] By manageable or reasonable sized, we mean a size of the representation that can be stored in the memory and efficiently manipulated using present day computing resources. As such, what is reasonable is dependent on the computer in use.

For any logic function there are many Boolean factors and divisors. This poses a problem in choosing a good factor since there are so many factors. If the domain is restricted to a particular subset of expressions, then the division operation is unique and much easier to carry out. A restricted version of division is called algebraic division which we describe in the next section.

7.3 Algebraic Division

We begin the description of algebraic division with some definitions. The *support* of f denoted as $sup(f)$ is the set of all variables v that occur in f as v or $\overline{v}$. For example, if $f = a \cdot \overline{b} + c$, then $sup(f) = \{a, b, c\}$. We say that f is *orthogonal* to g, written as $f \perp g$, if $sup(f) \cap sup(g) = \phi$. For example, $f = a + b$ and $g = c + d$ are orthogonal.

The function g is an *algebraic divisor* of f if there exist h and r such that $f = g \cdot h + r$, where $h \neq 0$, and $g \perp h$, and r has as few cubes as possible. The function g divides f *evenly* if $f = g \cdot h$, where $h \neq 0$, $g \perp h$ and $r = \phi$. The *quotient* f/g is the unique h such that $f = g \cdot h + r$, i.e., $f = (f/g) \cdot g + r$.

In the next section, we consider the two main problems of algebraic optimization, namely computing f/g given f and g, and determining divisors g of a given function f.

7.3.1 Computing the Quotient

Given that f and g are lists of cubes, with $f = \{b_1, b_2, \ldots, b_{|f|}\}$ and $g = \{a_1, a_2, \ldots, a_{|g|}\}$. Define $h_i = \{c_j | a_i \cdot c_j \in f\}$ for all $i = 1, 2, \ldots, |g|$, i.e., h_i corresponds to all the multipliers of the cube a_i in g that produce elements of f. It is easy to see that

$$f/g = \bigcap_{i=1}^{|g|} h_i = h_1 \cap h_2 \ldots \cap h_{|g|}$$

As an example, consider:

$$\begin{aligned} f &= a \cdot b \cdot c + a \cdot b \cdot d + d \cdot e \\ g &= a \cdot b + e \end{aligned}$$

We have $|g| = 2$ and $|f| = 3$. The number of comparison is $3 \times 2 = 6$, and they give:

$$h_1 = \{c, d\}$$

Therefore, $h_1 \cap h_2 = d$, and

$$f = (a \cdot b + e) \cdot d + a \cdot b \cdot c$$

The above strategy requires $O(|f|\,|g|)$ operations. Encoding and sorting the cubes of f and g can reduce the complexity to $O((|f| + |g|)log(|f| + |g|))$ [30].

7.3.2 Kernels and Algebraic Divisors

Given an efficient method for algebraic division, optimization can be carried out if good algebraic divisors can be found. The set of algebraic divisors is defined as $D(f) = \{g | f/g \neq \phi\}$. The *primary divisors* of f are defined as $P(f) = \{f/c | c \text{ is a cube}\}$. For example, if

$$f = a \cdot b \cdot c + a \cdot b \cdot d \cdot e$$

then

$$f/a = b \cdot c + b \cdot d \cdot e$$

is a primary divisor.

Proposition 7.3.1 *Every divisor of f is contained in a primary divisor, i.e., if g divides f, then $g \subseteq p \in P(f)$.*

Proof. Let $c \in f/g$ be a cube. Then $g \subseteq f/(f/g)$ and $f/(f/g) \subseteq f/c \in P(f)$. □

A function g is termed *cube-free* if the only cube that divides g evenly is 1. The *kernels* of f are defined as $K(f) = \{k | k \in P(f), k \text{ is cube-free}\}$. For example, if

$$f = a \cdot b \cdot c + a \cdot b \cdot d \cdot e$$

then

$$f/a = b \cdot c + b \cdot d \cdot e$$

is a primary divisor but not cube-free since b is a factor of f/a.

$$f/a = b \cdot (c + d \cdot e)$$

However, $f/(a \cdot b) = c + d \cdot e$ is a kernel. $a \cdot b$ is called a *cokernel*. Note that by definition, cokernels are always cubes.

The following theorem (originally proven in [15]) is the basis of algebraic optimization methods

Theorem 7.3.1 *f and g have a nontrivial common divisor d (i.e., d is not a cube) if and only if there exist kernels $k_f \in K(f)$ and $k_g \in K(g)$ such that $k_f \cap k_g$ is nontrivial with 2 or more terms (not a cube).*

Proof. It is clear that $k_f \cap k_g$ is a common divisor. It remains to prove the "only if" part.

Let d divide f, and let d divide g with d being nontrivial with 2 or more terms. Then there is a cube-free e such that e divides d. Now e divides f so by Proposition 7.3.1, $e \subseteq k_f \in P(f)$ and $e \subseteq k_g \in P(g)$ for some k_f and k_g. Since e is cube-free, k_f and k_g are as well. Hence, $k_f \in K(f)$ and $k_g \in K(g)$. Finally, since $e \subseteq k_f \cap k_g$, we have $k_f \cap k_g$ being nontrivial. □

We can therefore use the kernels of f and g to locate common divisors. Note that these are not the only common divisors of f and g, but they are good common divisors to consider during logic optimization. We compute the set of kernels for each logic expression, then form nontrivial intersections among kernels from the different logic expressions. If this intersection set is empty, then by Theorem 7.3.1, we need only look for divisors consisting of single cubes. If we find a nontrivial intersection, then we have found a algebraic divisor common to two or more expressions.

7.3.3 Computing the Kernels

The kernels of a function f can be computed using the algorithm of Figure 7.1.

The kernel generation algorithm first makes f cube-free by finding its largest cube factor. It then selects the literals of f in lexicographical order and divides them into f; the resulting quotient is a kernel if it is cube-free. (Note that this kernel might contain other kernels too.) If it is not cube-free, then it is made cube-free by selecting its largest cube factor. Note that in this context the largest cube is the cube with the most number of literals. The procedure is repeated on the resulting functions until functions with no kernels (called the level-0 kernels of f) are found. A major efficiency is obtained by noting that if the largest cube factor extracted contains an already selected literal, then the current branch can be terminated, since all the kernels that can be found by continuing have already


```
KERNELS(f):
{
    c_f = largest cube (with maximum number of literals) factor of f ;
    K = KERNEL1(0, f/c_f) ;
    if (f is cube-free)
        return(f ∪ K) ;
    return(K) ;
}

KERNEL1(j, g):
{
    R = g ;
    N = Maximum index of variables in g ;
    for(i = j + 1; i ≤ N; i = i + 1) {
        if (l_i in 1 or no cubes of g) continue ;
        c = largest cube dividing g/l_i evenly ;
        if (for all k ≤ i, l_k ∉ c) /* Pruning Condition */
            R = R ∪ KERNEL1(i, g/(l_i ∩ c)) ;
    }
    return(R) ;
}
```

Figure 7.1: Procedure to determine all the kernels of a single-output logic function

been generated. This leads to an algorithm in which no cokernel is duplicated.

As an example consider:

$$f = a \cdot b \cdot c \cdot d + a \cdot b \cdot c \cdot e + a \cdot b \cdot e \cdot f$$

In the routine **KERNELS** $c_f = a \cdot b$. Therefore,

$$f/c_f = c \cdot d + c \cdot e + e \cdot f$$

In the next step we call **KERNEL1**$(0,\ c \cdot d + c \cdot e + e \cdot f)$.

In **KERNEL1** we set $R = \{c \cdot d + c \cdot e + e \cdot f\}$. Since the ordering is lexicographic, we have $l_1 = a$, $l_2 = b$, etc. Note that $N = 6$. The literals l_1 and l_2 are in none of the terms of R, and we

move to $l_3 = c$. The largest cube dividing $(c \cdot d + c \cdot e + e \cdot f)/c$ which is $d + e$ is 1.

We therefore make a recursive call to **KERNEL1**(3, $(c \cdot d + c \cdot e + e \cdot f)/(c \cap 1)$). This call returns with $\{d + e\}$. In the parent **KERNEL1** R is set to $\{c \cdot d + c \cdot e + e \cdot f,\ d + e\}$. We skip $l_4 = d$ and move to $l_5 = e$. The largest cube evenly dividing $(c \cdot d + c \cdot e + e \cdot f)/e$ which is $c + f$ is 1. We next call **KERNEL1**(5, $(c \cdot d + c \cdot e + e \cdot f)/(e \cap 1)$). This returns with $c + f$.

We end with $K = R = \{c \cdot d + c \cdot e + e \cdot f,\ d + e,\ c + f\}$.

A second example illustrates the pruning condition: If the largest cube factor extracted contains an already selected literal, then the current branch can be terminated, since all kernels that can be found by continuing have already been generated. Consider:

$$f = a \cdot b \cdot c \cdot (d + e) \cdot (k + l) + a \cdot f \cdot g + h$$

In the first call to **KERNEL1**, we will generate the kernels corresponding to:

$$f/a = b \cdot c \cdot (d + e) \cdot (k + l) + f \cdot g$$

KERNEL1 calls itself recursively to compute

$$f/(a \cdot b) = c \cdot (d + e) \cdot (k + l)$$

Since $f/(a \cdot b)$ is not cube-free, the next recursive call to **KERNEL1** will use $(d + e) \cdot (k + l)$. All the kernels of this expression will be generated.

We move up one level in the recursion and compute

$$f/(a \cdot c) = b \cdot (d + e) \cdot (k + l)$$

At this stage, we note that $f/(a \cdot c)$ is not cube-free, and the largest cube dividing this expression evenly is b. However, b is an already selected literal implying that we have already generated the kernels for the cube-free expression $(d + e) \cdot (k + l)$. We do not have to recursively call **KERNEL1** for this branch and can go ahead to $f/(a \cdot d)$.

It is possible to modify the **KERNEL1** procedure to generate only the level-0 kernels which do not contain other kernels. This modification is based on the observation that if no kernels of g are found in the **for** loop, then g is a level-0 kernel.


```
GFACTOR(f):
{
    if (number of terms in f is 1)
        return(f) ;
    g = CHOOSE_DIVISOR(f) ;
    (h, r) = DIVIDE(f, g) ;
    f = GFACTOR(g) · GFACTOR(h) + GFACTOR(r) ;
    return(f) ;
}
```

Figure 7.2: Procedure to algebraically factor a function

7.3.4 Factoring Algorithm

A function can be algebraically factored using the notions presented in previous sections. A generic factoring algorithm is shown in Figure 7.2.

The procedure **DIVIDE** performs algebraic division as described in Section 7.3.1, and reexpresses f as $g \cdot h + r$. The procedure **CHOOSE_DIVISOR** is critical to obtaining a good factorization. One alternative is to select an arbitrary level-0 kernel as a divisor. This may not produce the best final result. Another alternative is to select a kernel which when substituted into the original function maximally reduces the total number of literals. For example, given:

$$X = a \cdot c + a \cdot d + a \cdot e + a \cdot g + b \cdot c + b \cdot d + b \cdot e + b \cdot f + c \cdot e + c \cdot f + d \cdot f + d \cdot g$$

if, in the procedure **CHOOSE_DIVISOR**, we choose literals in lexicographical order we obtain:

$$X = a \cdot (c + d + e + g) + b \cdot (c + d + e + f) + c \cdot (e + f) + d \cdot (f + g)$$

However, if we choose kernels, we obtain a better factorization with fewer literals:

$$X = (c + d + e) \cdot (a + b) + f \cdot (b + c + d) + g \cdot (a + d) + c \cdot e$$

7.3.5 Extraction and Resubstitution Algorithm

To identify cube-free expressions that occur in several functions $\{f_i\}$ we do the following:

1. Generate kernels for each f_i.
2. Select a pair of kernels $k_1 \in K(f_i)$ and $k_2 \in K(f_j)$ where $i \neq j$ such that $k_1 \cap k_2$ is not a cube. If no such pair exists, stop.
3. Set the new variable v to $k_1 \cap k_2$.
4. Update the associated functions to:

$$f_i = v \cdot (f_i/(k_1 \cap k_2)) + r_i$$

Common cubes are extracted as follows.

1. Select a pair of cubes $c_i \in f_i$, $c_2 \in f_j$ with $i \neq j$ such that $c_1 \cap c_2$ consists of 2 or more literals. If no such pair exists, stop.
2. Set the new variable x to $c_1 \cap c_2$.
3. Update each function f_i with the new variable wherever possible in the network.

Consider as an example the factored functions:

$$\begin{aligned} X &= a \cdot b \cdot (c \cdot (d + e) + f + g) + h \\ Y &= a \cdot i \cdot (c \cdot (d + e) + f + j) + k \end{aligned}$$

We have $d+e$ being a level-0 kernel of both functions, and extraction results in:

$$\begin{aligned} L &= d + e \\ X &= a \cdot b \cdot (c \cdot L + f + g) + h \\ Y &= a \cdot i \cdot (c \cdot L + f + j) + k \end{aligned}$$

Now, we select $c \cdot L + f + g$ as a level-0 kernel of the reexpressed X and $c \cdot L + f + j$ as a level-0 kernel of reexpressed Y. We obtain:

$$\begin{aligned} M &= c \cdot L + f \\ L &= d + e \\ X &= a \cdot b \cdot (M + g) + h \\ Y &= a \cdot i \cdot (M + j) + k \end{aligned}$$

Now X and Y have no kernel intersections that are nontrivial. We now extract common cubes. The cubes $a \cdot b \cdot M$ in X and $a \cdot i \cdot M$ in Y have 2 literals in common. Extraction produces:

$$\begin{aligned} N &= a \cdot M \\ M &= c \cdot L + f \\ L &= d + e \\ X &= b \cdot (N + a \cdot g) + h \\ Y &= i \cdot (N + a \cdot j) + k \end{aligned}$$

Because we are continually recomputing level-0 kernels on the reexpressed functions, it is possible to obtain decompositions corresponding to level-k > 0 kernels. If we collapse L into M into N above, we obtain:

$$\begin{aligned} N &= a \cdot (c \cdot (d + e) + f) \\ X &= b \cdot (N + a \cdot g) + h \\ Y &= i \cdot (N + a \cdot j) + k \end{aligned}$$

where N contains a level-1 kernel of the original X and Y, since it contains the level-1 kernel M which contains the level-0 kernel $d+e$.

7.3.6 Algebraic Resubstitution with Complement

Algebraic factorization and resubstitution can be performed with the complement of a given divisor. For example, consider:

$$f = a \cdot b + a \cdot c + \overline{b} \cdot \overline{c} \cdot d$$

where we choose $b + c$ as a level-0 kernel of f and decompose f as shown below.

$$\begin{aligned} f &= a \cdot X + \overline{b} \cdot \overline{c} \cdot d \\ X &= b + c \end{aligned}$$

In many cases it is useful to check if the complement of the new variable is an algebraic divisor for the function. In this case we can obtain:

$$\begin{aligned} f &= a \cdot X + \overline{X} \cdot d \\ X &= b + c \end{aligned}$$

7.4 Rectangles and Rectangle Covering

One of the key problems in algebraic optimization is the identification

determining a good set of divisors for algebraic factoring, decomposition, and extraction. The problem of finding a kernel and finding a single-cube or multiple-cube divisor can be reduced to the combinatorial optimization problem of rectangle covering. This formulation of the problem is not only elegant, but it is also favors the development of fast and effective algorithms. In this section, we closely follow the methods and algorithms of [36].

7.4.1 Definitions

A *rectangle* (R, C) of a matrix B, $B_{ij} \in \{0, 1, *\}$ is a subset of rows R and subset of columns C such that $B_{ij} \in \{1, *\}$ for all $i \in R$ and $j \in C$. Note that the rows and columns forming the rectangle do not have to be contiguous.

A rectangle (R_1, C_1) is said to *strictly contain* rectangle (R_2, C_2) if $R_2 \subseteq R_1$ and $C_2 \subset C_1$ or $R_2 \subset R_1$ and $C_2 \subseteq C_1$.

A rectangle (R, C) of B is said to be a *prime rectangle* if it is not strictly contained in any other rectangle of B.

The *corectangle* of a rectangle (R, C) is the pair (R, C') where C' is the set of columns not in C.

A set of rectangles $(\{R^K, C^K\})$ form a rectangle cover of matrix B if $B_{ij} = 1$ implies that $i \in R^k$ and $j \in C^k$ for some k. Thus, each 1 in B must be covered by at least one rectangle from the cover. A covering need not be disjoint, and therefore a 1 in B can be covered by more than one rectangle. The points of B which are labeled $*$ are not required to be covered by any rectangle in the cover and therefore represent don't-care points in the matrix.

In the following matrix

	1	2	3	4	5
1	1	1	1	0	0
2	1	*	1	0	*
3	0	1	1	0	1
4	1	0	1	1	1

$(\{1, 2\}, \{2, 3\})$ is a rectangle, but it is not prime as it is contained in the prime rectangle $(\{1, 2\}, \{1, 2, 3\})$. $(\{2, 4\}, \{1, 3, 5\})$ is another prime rectangle while $(\{2, 3\}, \{1, 2\})$ is not a rectangle.

Each rectangle (R^k, C^k) has an associated weight or cost defined by a weight function $w(R^k, C^k)$. The weight of a rectangle cover is then defined as the sum:

$$\sum_k w(R^k, C^k)$$

The minimum weighted rectangle covering problem is that of finding a rectangle cover of a matrix with minimum total weight.

7.4.2 Rectangles and Kernels

Rectangles provide an alternate way of looking at kernels of a function. Consider the expression $g = a \cdot b \cdot e + a \cdot c \cdot d + b \cdot c \cdot d$. This Boolean expression can be represented using a *cube-literal matrix* shown below where each row corresponds to a cube in the expression and the columns correspond to all the distinct literals.

	a	b	c	d	e
$a \cdot b \cdot e$	1	1	0	0	1
$a \cdot c \cdot d$	1	0	1	1	0
$b \cdot c \cdot d$	0	1	1	1	0

Consider the prime rectangle $(R, C) = (\{2, 3\}, \{3, 4\})$ and its corectangle $(R, C') = (\{2, 3\}, \{1, 2, 5\})$. The rectangle obviously corresponds to a cube $c \cdot d$ that is common to all the product terms corresponding to rows in R. Since the rectangle is prime, it is the largest cube common to all the product terms in R. If this cube is extracted from these product terms, the resulting expression is cube-free and is also a divisor of the original function g. In other words, the resulting expression is a kernel of g. The expression resulting from the extraction of the cube corresponds to the corectangle $(R, C') = (\{2, 3\}, \{1, 2, 5\})$, which is $a + b$. Therefore, each prime rectangle is a cokernel, while each corectangle of a prime rectangle is a kernel of the expression represented by the matrix.

From the rectangle interpretation of kernels, it is also possible to understand more clearly the notion of the level of a kernel. A level-0 kernel is the corectangle of a prime rectangle which has no other rectangle containing its column set, i.e., a rectangle of maximal width. The corectangle of a prime rectangle of maximal height, i.e., one whose row set is not contained in any other rectangle, corresponds to a kernel of maximal level.

7.4.3 Common-Cube Extraction

Common-cube extraction is the process of finding cubes common to two or more expressions and extracting the common cube to simplify each of the expressions. To optimize the network it is necessary to find the particular cubes to introduce that provide an optimal decomposition. The optimal decomposition can be defined as minimizing the total number of literals summed over all expressions or minimizing the total number of literals given a bound on the number of levels of logic in the final circuit.

Common cubes can be easily identified using the cube-literal matrix described above. Consider the following equations:

$$\begin{aligned} F &= a \cdot b \cdot c + a \cdot b \cdot d + e \cdot g \\ G &= a \cdot b \cdot f \cdot g \\ H &= b \cdot d + e \cdot f \end{aligned}$$

The cube-literal matrix for this expression is:

		a	b	c	d	e	f	g
F_1	$a \cdot b \cdot c$	1	1	1	0	0	0	0
F_2	$a \cdot b \cdot d$	1	1	0	1	0	0	0
F_3	$e \cdot g$	0	0	0	0	1	0	1
G_1	$a \cdot b \cdot f \cdot g$	1	1	0	0	0	1	1
H_1	$b \cdot d$	0	1	0	1	0	0	0
H_2	$e \cdot f$	0	0	0	0	1	1	0

The rectangle $(\{1,2,4\},\{1,2\})$ corresponds to common cube ab which is present in functions F and G. If this common cube is extracted as a new function X, the equations can be rewritten as:

$$\begin{aligned} F &= X \cdot c + X \cdot d + e \cdot g \\ G &= X \cdot f \cdot g \\ H &= b \cdot d + e \cdot f \\ X &= a \cdot b \end{aligned}$$

The process of extracting a cube modifies a Boolean network. A new node is added to the Boolean network with a logic function which is the common-cube divisor. All functions which the cube divides are replaced with the algebraic division of the function by the single cube. In order to extract cubes efficiently in an iterative algorithm it is necessary to modify the cube-literal matrix incrementally to reflect the extraction of the cube. The advantage is that

the cube-literal matrix does not have to be recreated as each cube is extracted.

The modifications required to form the new cube-literal matrix are the following. A new row is added to reflect the new single cube expression added to the network. The entries covered by the rectangle are marked with a $*$ to reflect that the position has been covered. However, the $*$ allows other rectangles to cover the same position.

The choice of the weight function for a rectangle measures the optimization goal for cube extraction. To minimize the total number of literals in the network the weight of a rectangle is chosen so that the weight of a rectangle cover of the cube-literal matrix equals the total number of literals in the network after the new single-cube functions are added to the network. Hence, a minimum weighted rectangle cover corresponds to the optimal simultaneous extraction of a collection of cubes. The weight of a rectangle is defined as:

$$w(R,C) = \begin{cases} |C| & if \quad |R| = 1 \\ |C| + |R| & if \quad |R| > 1 \end{cases}$$

If there is a single row in the rectangle, then it corresponds to leaving the cube unchanged in the network. Hence, the weight of the rectangle counts the number of literals in the cube, which equals the number of columns. When the number of rows is greater than one, this corresponds to creating a new single cube function with $|C|$ literals and substituting this new function into $|R|$ other cubes at a cost of $|R|$ literals.

Note that the above weight does not reflect the savings obtained in terms of the number of literals by extracting a common cube. Therefore, when searching for a cube to extract it is useful to define a second function called the *value* of the rectangle. For cube extraction the value of the rectangle should indicate the savings obtained from extracting the corresponding cube. Since the number of literals before cube extraction is the number of 1s in the rectangle and the number of literals after cube extraction is the weight of the rectangle, the value $v(R,C)$ of a rectangle is defined as:

$$v(R,C) = |\{(i,j)|B_{ij} = 1, i \in R, j \in C\}| - w(R,C)$$

For example, for the rectangle $(\{1,2,4\},\{1,2\})$ in the cube-literal matrix shown above, its weight is the number of rows plus the number of columns, which equals 5. There are 6 positions in

this rectangle and each of them has a 1. Therefore, the value of the rectangle is $6 - 5 = 1$. Therefore only one literal can be saved by extracting this rectangle, as illustrated in the example above.

7.4.4 Kernel Intersection

As described previously, intersections among the kernels of a collection of expressions are useful for finding common multiple-cube divisors between two or more expressions. If two functions share a common multiple-cube divisor, then the common divisor can be found as the intersection of a kernel from each of the functions.

The Boolean matrix associated with the optimal kernel intersection problem is called the *cokernel-cube matrix*. A row in this matrix corresponds to a cokernel (and its associated kernel) and each column corresponds to a cube present in some kernel, called a *kernel-cube*. The entry B_{ij} is set to 1 if the kernel associated with row i contains the cube associated with column j.

Consider the functions:

$$\begin{aligned} F &= a \cdot f + b \cdot f + a \cdot g + c \cdot g + a \cdot d \cdot e + b \cdot d \cdot e + c \cdot d \cdot e \\ G &= a \cdot f + b \cdot f + a \cdot c \cdot e + b \cdot c \cdot e \\ H &= a \cdot d \cdot e + c \cdot d \cdot e \end{aligned}$$

The kernels and cokernels of each of the functions are shown below:

Function	Cokernel	Kernel
F	a	$d \cdot e + f + g$
F	b	$d \cdot e + f$
F	$d \cdot e$	$a + b + c$
F	f	$a + b$
F	c	$d \cdot e + g$
F	g	$a + c$
G	a	$c \cdot e + f$
G	b	$c \cdot e + f$
G	f	$a + b$
G	$c \cdot e$	$a + b$
H	$d \cdot e$	$a + c$

Note that functions F and G are themselves kernels but have not been shown above for the ease of presentation. Let us number the cubes in the original function from 1 to 13, with $a \cdot f$ being 1, $b \cdot f$ being 2, and so on. The cokernel-cube matrix for this set of kernels

is shown below. Note that instead of 1s in the matrix, we have numbers. These numbers indicate a cube of the original functions formed by multiplying the cokernel corresponding to a row and the cube corresponding to a column. For example, in the third row under column a we have the number 5 corresponding to the the fifth cube $a \cdot d \cdot e$.

		a	b	c	ce	de	f	g
F	a	0	0	0	0	5	1	3
F	b	0	0	0	0	6	2	0
F	$d \cdot e$	5	6	7	0	0	0	0
F	f	1	2	0	0	0	0	0
F	c	0	0	0	0	7	0	4
F	g	3	0	4	0	0	0	0
G	a	0	0	0	10	0	8	0
G	b	0	0	0	11	0	9	0
G	f	8	9	0	0	0	0	0
G	$c \cdot e$	10	11	0	0	0	0	0
H	$d \cdot e$	12	0	13	0	0	0	0

A rectangle of the cokernel-cube matrix identifies an intersection of kernels. The columns of the rectangle identify the cubes in the subexpression, and the rows in the rectangle identify the particular functions the subexpression divides. For example, rectangle $(\{3, 4, 9, 10\}, \{1, 2\})$ identifies the subexpression $a + b$. This corresponds to the factorization of the equations into the form:

$$\begin{aligned} F &= d \cdot e \cdot X + f \cdot X + a \cdot g + c \cdot g + c \cdot d \cdot e \\ G &= c \cdot e \cdot X + f \cdot X \\ H &= a \cdot d \cdot e + c \cdot d \cdot e \\ X &= a + b \end{aligned}$$

Whenever a new subexpression is identified, it is inserted into the Boolean network. This consists of adding a new node to the network and dividing the node into each of the expressions which this node divides. A new cokernel-cube matrix is then created for the modified Boolean network.

To reduce the complexity of extracting each factor from the network it is desirable to modify the cokernel-cube matrix incrementally as each subexpression is identified. To do this new rows are added to the cokernel-cube matrix for each kernel of the new subexpression. The cubes which are formed by the insertion of this new

factor into the network are then marked as covered. This includes the points directly contained in the rectangle and other points which are labeled with the same number. These points are marked $*$ so that other rectangles can cover them.

The weight of a rectangle of the cokernel-cube matrix is chosen to reflect the number of literals in the network if the corresponding common subexpression is inserted into the network. A minimum weighted rectangle cover of the cokernel-cube matrix then corresponds to a simultaneous selection of a set of subexpressions to add to the network in order to minimize the total number of literals.

Let w_j^c be the number of literals in the kernel-cube for column j. w_j^c is also called the column weight of column j. If a rectangle (R, C) is used to identify a subexpression, then a new function is formed from the columns of C. This new function has $\sum_{j \in C} w_j^c$ literals. Let w_i^r be 1 plus the number of literals in the cokernel corresponding to row i. w_i^r is also called the row weight of row r. The chosen subexpression divides the expressions indicated by the rows R of the rectangle. After algebraic division by the subexpression each of these expressions consists of a sum of the corresponding cokernel cubes multiplying the literal for the new expression. The number of literals in the affected functions after the extraction of the subexpression corresponding to the rectangle is $\sum_{i \in R} w_i^r$. Therefore, the weight of a rectangle (R, C) in the cokernel-cube matrix is defined as:

$$w(R, C) = \sum_{i \in R} w_i^r + \sum_{j \in C} w_j^c$$

The value of a rectangle measures the difference in the number of literals in the network if the particular rectangle is selected. The number of literals after the rectangle is selected is the weight of the rectangle as defined above. Let V_{ij} be the number of literals in the cube which is covered by position (i, j) of the cokernel-cube matrix. Then the number of literals before extraction of the rectangle is simply $\sum_{i \in R, j \in C} V_{ij}$. As elements of the cokernel-cube matrix are covered, their values V_{ij} are set to 0. This includes the elements V_{ij} covered by the matrix and all other elements which represent the same cube in the network. The value of a rectangle (R, C) of the cokernel-cube matrix is thus defined as:

$$v(R, C) = \sum_{i \in R, j \in C} V_{ij} - w(R, C)$$

For example, for the rectangle $(\{3, 4, 9, 10\}, \{1, 2\})$ in the cokernel-cube matrix above, $\sum_{i \in R, j \in C} V_{ij} = 3+3+2+2+2+2+3+3 = 20$, $\sum_{i \in R} w_i^r = 3+2+2+3 = 10$, $\sum_{j \in C} w_j^c = 1+1 = 2$. Therefore, the value of the rectangle is $value = 20 - 10 - 2 = 8$. Eight literals can be saved by extracting the expression corresponding to the rectangle, as can be verified in the example above.

7.4.5 Rectangle Algorithms

There are two types of algorithms for rectangle covering. The first type of algorithm is greedy and selects one rectangle at a time and modifies the matrix to reflect the extraction of the rectangle. The advantage of this technique is that it immediately takes into account common factors between the newly extracted function and the rest of the logic network. The disadvantage of this approach is that it selects only one rectangle at a time and does not easily account for the simultaneous extraction of multiple rectangles. The second type of algorithm finds the best collection of factors to extract at each step by solving the minimum-weighted rectangle covering problem heuristically. First, all the prime rectangles are generated, and a collection of rectangles are then extracted. Second, the matrix is updated, and the entire process is repeated to find factors between the new expressions and the remainder of the logic network.

In this section we will first discuss a heuristic algorithm called **PING-PONG** that can be used to generate a good valued rectangle without generating all prime rectangles. Such an algorithm is useful for the extraction of a single good factor.

The inputs to the algorithm are the matrix and the value function which computes the value of a rectangle. The value function is defined in terms of the row and column weights. The algorithm chooses two seed rectangles — one corresponding to the best row and the other corresponding to the best column. These two seed rectangles are expanded until they are prime. From these expanded rectangles the one with the larger value is returned.

We will illustrate this algorithm with an example. Consider the function

$$F = a \cdot c + a \cdot d + a \cdot e + a \cdot g + b \cdot c + b \cdot d$$
$$+ b \cdot e + b \cdot f + c \cdot e + c \cdot f + d \cdot f + d \cdot g$$

Its kernels (and cokernels) are $c + d + e + g(a)$, $c + d + e + f(b)$, $a + b + e + f(c)$, $a + b + f + g(d)$, $a + b + c(e)$, $b + c + d(f)$, and $d(g)$. The cokernel-cube matrix with the value at each position is shown

below. Note each row has a weight of 2 (because each cokernel has one literal) and each column has a weight of 1 (as there is one literal in each kernel-cube).

	a	b	c	d	e	f	g
a	0	0	2	2	2	0	2
b	0	0	2	2	2	2	0
c	2	2	0	0	2	2	0
d	2	2	0	0	0	2	2
e	2	2	2	0	0	0	0
f	0	2	2	2	0	0	0
g	0	0	0	2	0	0	0

We will illustrate the **PING-PONG** procedure using one of the seed rectangles, the best row. The dual of the procedure with the role of rows and columns interchanged has to be followed for the other seed rectangle, the best column. In this example there are four rows with the same value, and we arbitrarily choose row 1 as the seed. Therefore, the seed rectangle is $(\{1\}, \{3,4,5,7\})$ with a value 2. This value is derived by adding the numbers at all points in the rectangle $(2 \times 4 = 8)$ and subtracting the row weight (2) and the column weights $(1 \times 4 = 4)$ from it $(8 - 4 - 2 = 2)$. We now try to add another row to this rectangle by examining all the other rows and selecting a row which if added maximizes the value of the seed rectangle. This row is added, and only columns that are both in the seed rectangle column set and the row selected are retained. In this example addition of row 2 maximizes the value of the rectangle to 5, and therefore the row is added to the seed rectangle. The new rectangle is $(\{1,2\}, \{3,4,5\})$. Note that column 7 was deleted from the initial rectangle and row 2 was added to form this new rectangle. A new row is selected for addition to this rectangle. This time it is row number 6, which produces a rectangle $(\{1,2,6\}, \{3,4\})$ with value 4. This process is repeated until the column set of the rectangle consists of only one column. This happens when row 5 is added to the set, producing the rectangle $(\{1,2,5,6\}, \{3\})$ with a value -1. From all the rectangles produced in the process the best one, which is $(\{1,2\}, \{3,4,5\})$, is chosen.

This rectangle can now be improved. One column of this rectangle with the largest value is chosen as a seed column. In this case it is column 3. Now the dual of the procedure described above is followed with the role of rows and columns interchanged, i.e., columns are added to this seed column to form new rectangles. Each time,

a column that maximizes the value of the rectangle is chosen, and only rows that are both in the row set of the rectangle and the column chosen are retained. The process continues until there is only one row left in the rectangle. The rectangle with the largest value (including the initial one) is returned. In this example the successive rectangles that are produced are $(\{1,2,5,6\}, \{3\})$ with value -1, $(\{1,2,6\}, \{3,4\})$ with value 4, $(\{1,2\}, \{3,4,5\})$ with value 5, and $(\{1\}, \{3,4,5,7\})$ with value 2. The best rectangle is still the initial one, which is $(\{1,2\}, \{3,4,5\})$.

If the best rectangle chosen changed, then another pass is initiated where a row of the best rectangle is chosen as the seed row and is grown into a rectangle using the method described previously. This method of growing seed rows and columns is iterated until no improvement can be obtained. In this example the best rectangle chosen would be $(\{1,2\}, \{3,4,5\})$, corresponding to the kernel intersection $c + d + e$.

We will now illustrate a method to generate all prime rectangles given a matrix. The worst-case complexity of this method is exponential in the size of the matrix. However, when the matrix is sparse, it is often feasible to enumerate all prime rectangles. We use as an example the expression

$$F = a \cdot b \cdot c \cdot d \cdot g + a \cdot b \cdot c \cdot d \cdot h + a \cdot b \cdot c \cdot e + a \cdot b \cdot c \cdot f + a \cdot b \cdot i$$

whose cube-literal matrix is shown below.

	a	b	c	d	e	f	g	h	i
1	1	1	1	1	0	0	1	0	0
2	1	1	1	1	0	0	0	1	0
3	1	1	1	0	1	0	0	0	0
4	1	1	1	0	0	1	0	0	0
5	1	1	0	0	0	0	0	0	1

The first step in generating prime rectangles is to process and record the trivial prime rectangles. A single row (column) is a prime rectangle if it is not contained in any other row (column). Also, if the matrix does not contain any columns (rows) with all 1s, then the rectangle consisting of all the rows (columns) and no columns (rows) is also prime. In our example above we have no trivial prime rectangles.

The recursive procedure described in Figure 7.3 is qualitatively similar to the kernel generation procedure of Figure 7.1. It

```
GENERATE_RECTANGLES(M):
{
    Find all trivial rectangles and store ;
    rect = new rectangle ;
    RECTANGLE1(M, 0, rect) ;
}

RECTANGLE1(M, index, rect):
{
    for( each column c of M ) {
        if ( c has 2 or more elements and c.index ≥ index ) {
            M1 = new matrix ;
            for( each element p in column c )
                copy row corresponding to p in M to M1 ;

            rect1 = new rectangle ;
            Duplicate columns of rect in rect1 ;
            Rows of rect1 correspond to elements of c in M ;

            if ( indices of all columns of all 1s in M1 ≥ c.index ) {
                Delete columns of all 1s in M1 and add to rect1 ;
                Store rect1 in prime rectangle set ;
                RECTANGLE1( M1, c.index, rect1) ;
            }
        }
    }
}
```

Figure 7.3: Procedure to determine all the prime rectangles of a matrix

keeps track of the current submatrix which is being searched for prime rectangles, namely M, the current column index, namely *index*, and the rectangle found up to this point, namely *rect*. The recursive assumption made is that all the rows of *rect* contain a 1 or $*$ for all columns of *rect*.

The procedure will search the submatrix M to find all of the prime rectangles with fewer rows but more columns. Each column c

with an index greater than the starting index is examined as a column to include in the rectangle. If the column has only a single element, then it cannot create a nontrivial rectangle, so only columns with two or more elements are of interest. A submatrix M_1 of the original matrix M is created by selecting only the rows where the column c has a nonzero value, and a new rectangle $rect_1$ is formed from the columns of the old rectangle and the rows for which c has a nonzero value.

Any column of the submatrix M_1 including c which is now all 1s can also be added to the rectangle $rect_1$. At this point, $rect_1$ represents a new prime rectangle of the matrix, and M_1 is a new submatrix to be searched for more prime rectangles.

A pruning operation can be performed when $rect_1$ is constructed. If a column of 1s occurs for a column index less than the starting index, then all rectangles in the current submatrix M_1 have already been examined when that column index was processed. Hence, if this condition is detected, it is not necessary to do another recursion. This condition is equivalent to the pruning condition in Figure 7.1.

In our example we begin in **RECTANGLE1** with the original matrix M, the starting column index 0, and with *rect* being a new rectangle. We pick column 1 in the loop. There are more than two elements in column 1 so we create a new matrix M_1. M_1 will be set to M since column 1 has all five elements. $rect_1$ is created as a new rectangle and is initially set to $(\{1,2,3,4,5\},\{1\})$. Next, column 2 which has all 1s in M_1 is added to $rect_1$ since its index is greater than 1. We now have a prime rectangle $(\{1,2,3,4,5\},\{1,2\})$ that is added to the set of prime rectangles. This rectangle corresponds to the kernel (cokernel) $c \cdot d \cdot g + c \cdot d \cdot h + c \cdot e + c \cdot f + i$ $(a \cdot b)$.

We enter the recursive call of **RECTANGLE1** with M shown below:

	c	d	e	f	g	h	i
1	1	1	0	0	1	0	0
2	1	1	0	0	0	1	0
3	1	0	1	0	0	0	0
4	1	0	0	1	0	0	0
5	0	0	0	0	0	0	1

The variable *index* in this call is 1 and *rect* is $(\{1,2,3,4,5\}, \{1,2\})$. We first pick column 3 in M and create M_1 which is M

but with row 5 deleted. $rect_1$ is set to $(\{1,2,3,4\},\{1,2\})$. We add the column of all 1s, namely column 3, to $rect_1$ to make it a prime rectangle. The prime rectangle $(\{1,2,3,4\},\{1,2,3\})$ is added to the set. This rectangle corresponds to the kernel (cokernel) $d{\cdot}g{+}d{\cdot}h{+}e{+}f$ $(a \cdot b \cdot c)$.

In the next recursive call we will discover the prime rectangle $(\{1,2\},\{1,2,3,4\})$ corresponding to the kernel (cokernel) $g+h$ $(a \cdot b \cdot c \cdot d)$. We have now discovered all the kernels of the expression.

Going back to the first call of **RECTANGLE1** we will pick column 2 with the original matrix M, the starting column index 0, and *rect* being a new rectangle. We create M_1 as a new matrix with all the rows and columns of M. $rect_1$ is set to $(\{1,2,3,4,5\},\{2\})$. When we check for the pruning condition we see that column 1 is a column of all 1s with an index less than 2, and we will not perform any recursive calls.

7.5 Boolean Division

So far we have primarily described algebraic optimization methods. In this section we will focus on Boolean division. As noted in Section 7.2 there are many Boolean divisors of a function. If a good Boolean divisor can be found, then two-level minimization under an appropriate don't-care set can be used to compute $f//g$, where we use // to denote Boolean division.

A procedure to compute $f//g$, given sum-of-products representations for f and g is given below. The procedure is restricted to the case where $sup(g) \subseteq sup(f)$.

1. Add an extra input to f corresponding to g. Call it G.
2. Construct a function F whose *DC*-set D is $G \cdot \overline{g} + \overline{G} \cdot g$, *ON*-set is $f \cap \overline{D}$, and *OFF*-set is $\overline{f \cup D}$.
3. Minimize F for minimum literals in sum-of-products form.

The specification of the don't-care set in the above procedure is key. The don't-care set corresponds to the set of inputs to f that cannot occur. Since G is itself a function of the primary inputs, G can never be a 0 when a minterm in the *ON*-set of g is applied. Similarly for the *OFF*-set.

Consider an example of the application of Boolean division.

$$\begin{aligned} f &= a \cdot b \cdot c + a \cdot \overline{b} \cdot \overline{c} + \overline{a} \cdot b \cdot \overline{c} + \overline{a} \cdot \overline{b} \cdot c \\ g &= a \cdot b + \overline{a} \cdot \overline{b} \end{aligned}$$

The specifications for the *DC*-set and *ON*-set for F are:

$$\begin{aligned} F^{DC} &= a \cdot b \cdot \overline{G} + \overline{a} \cdot \overline{b} \cdot \overline{G} + \overline{a} \cdot b \cdot G + a \cdot \overline{b} \cdot G \\ F^{ON} &= a \cdot b \cdot c \cdot G + \overline{a} \cdot \overline{b} \cdot c \cdot G + \overline{a} \cdot b \cdot \overline{c} \cdot \overline{G} + a \cdot \overline{b} \cdot \overline{c} \cdot \overline{G} \end{aligned}$$

Minimization produces:

$$F = c \cdot G + \overline{c} \cdot \overline{G}$$

Note that G has been used in true and complemented forms.

In general the above procedure works well, but the minimization at Step 3 is not guaranteed to result in a representation for F that uses the extra input G. In order to ensure that G appears in the minimized sum-of-products representation for F during heuristic two-level minimization using ESPRESSO, a modification is made. In the **EXPAND** step the literals in cubes corresponding to the inputs to F other than G are expanded first. In the **REDUCE** step the literals corresponding to the input G are reduced first.

There has been little work done in identifying Boolean divisors. A method based on multiple-valued minimization has been proposed [22]. Another technique that is used presently is to identify a good algebraic divisor and then use the Boolean division algorithm. This will produce at least as good results as algebraic division using the same divisor.

7.6 Don't-Care-Based Optimization

The don't-care conditions arising in multilevel logic can either be specified by the user or can be an artifact of the network structure. User specified don't-cares or don't-cares derived from considerations other than the network structure are called *external* don't-cares.

Internal don't-cares arise in multilevel logic because of the structure of a Boolean network. They are divided into satisfiability and observability don't-cares.